

Programación y Base de Datos

Unidad 1. Introducción a la programación y fundamentos para la formulación de algoritmos

Carlos Guillermo Chuncho Morocho

Carrera de Ingeniería Ambiental
Universidad Nacional de Loja

Formación académica

- Ing. Agrónomo (UNL, 2004).
- 2 maestrías previas: Cs. de la Agricultura (PUC Chile, 2017) y Agric. Ecológica (CATIE, 2011).
- Dipl. Sup. en Gestión de Proyectos (FLACSO, 2005).
- **Maestría en Estadística**, m. Ciencia de Datos e IA.

Herramientas de trabajo

Uso profesional habitual de **Visual Studio Code**, **Python**, **SQL** (PostgreSQL/PostGIS), **Git** y **Jupyter Notebook** en el análisis de datos ambientales y modelación de incendios forestales.

Investigación actual

Modelación probabilística y espacio-temporal de incendios forestales en Loja (jerárquica, Bernoulli, Matérn/SPDE). Publicaciones en *Fire Ecology* (2026), *Axioma* (2025) y *Rev. Est. Andaluces* (2024).

Este período dicto además: Matemática 2, Estadística Descriptiva, y Formulación del Proyecto de Titulación.

Carrera de Ingeniería Ambiental, Bloque 86 | carlos.chuncho@unl.edu.ec

Unidad 1 – Introducción a la programación

5 semanas ▪ 25,0 horas

1. Introducción a la programación: entorno de trabajo, sintaxis y errores.
2. Fundamentos de algoritmos: problema, algoritmo y programa.
3. Herramientas para el diseño de algoritmos: variables, expresiones y sentencias.
4. Aplicación de algoritmos a problemas ambientales sencillos.
5. Ingreso de datos y visualización de resultados.

Resultado de aprendizaje: conoce los elementos y estructuras básicas para el manejo de un lenguaje de programación, con capacidad de identificación y resolución de problemas sencillos en el contexto ambiental.

Unidad 2 – Condicionales e iteraciones. Funciones. Cadenas

6 semanas ▪ 30,0 horas

1. Condicionales: `if`, `elif`, `else`.
2. Funciones: definición, parámetros y valores de retorno.
3. Iteraciones: `for` y `while`.
4. Cadenas y operaciones con cadenas.
5. Listas y diccionarios.
6. Tuplas.

Resultado de aprendizaje: desarrolla y aplica algoritmos para el análisis de datos ambientales, con habilidades de análisis lógico y síntesis.

Unidad 3 – Manejo de archivos, datos y visualización

5 semanas ▪ 25,0 horas

1. Manejo de archivos: rutas, formatos `.txt` y `.csv`.
2. Bases de datos y SQL.
3. Transformación y agregación de dataframes (pandas).
4. Porciones indexadas de dataframes.
5. Creación y visualización de dataframes (matplotlib).

Resultado de aprendizaje: emplea herramientas básicas para el almacenamiento, manipulación y presentación de datos en proyectos sencillos.

Hoja de ruta

La hoja de ruta del curso

Proyecto integrador, equipos de trabajo, cronograma y evaluación

El proyecto integrador: un caso ambiental real

De qué se trata

Durante todo el semestre cada equipo trabaja con el mismo conjunto real de datos ambientales georreferenciados del cantón Loja (periodo 2019–2024) del proyecto **FIRELAB-Loja**: relieve, cobertura vegetal, clima e incendios forestales. En Programación y Base de Datos, cada bloque temático se resuelve construyendo programas cada vez más completos -de un primer script que lee datos a un flujo que los transforma, consulta y visualiza.

Contexto ambiental

25 estudiantes del curso se organizan en **6 equipos de 4 integrantes** (uno de 5), cada uno responsable de un bloque temático del dataset ambiental.

El proyecto tiene **3 partes, una por unidad** (formulación → metodología → resultados finales), y dentro de cada unidad se evalúa en las mismas **4 fases**: formulación, avance, producto técnico y defensa individual.

Software y herramientas que necesitaremos

Entorno de trabajo del curso

Todo el trabajo práctico se realiza en **Visual Studio Code (VS Code)** con el intérprete de **Python**, siguiendo el sílabo de la asignatura.

- **Visual Studio Code**: editor principal del curso.
- **Python** (versión reciente): lenguaje de programación del curso.
- Extensiones de VS Code para **Python** y **Jupyter Notebook**.
- Bibliotecas de datos: `pandas`, `matplotlib`, `sqlite3`.

Por ahora: instalen Visual Studio Code y sus dependencias (Python, extensiones, Git y GitHub) antes de la próxima sesión práctica, siguiendo el documento **"Guía de instalación y configuración del entorno"** entregado aparte.

Proyecto integrador: los 6 equipos y su reto

Equipos	Bloque temático (variables)	Pregunta guía en clave de programación
1 y 2	Relieve y accesibilidad: elevación, pendiente, orientación, distancia a asentamientos y vías.	¿Qué algoritmo clasifica automáticamente el grado de pendiente de cada celda del relieve?
3	Cobertura vegetal y uso del suelo: bosque nativo, páramo, área agropecuaria, plantación forestal, índices espectrales.	¿Qué programa calcula la proporción de cobertura vegetal por categoría a partir de los datos crudos?
4 y 5	Clima y condiciones atmosféricas: precipitación acumulada, días secos y húmedos, velocidad del viento.	¿Qué algoritmo cuenta los días secos y húmedos de un periodo a partir de los registros de precipitación?
6	Incendios forestales y riesgo: evidencia satelital de fuego y ocurrencia por celda, mes y año.	¿Qué programa identifica y cuenta los eventos de incendio por celda, mes y año?

Al ser más equipos que bloques temáticos, en dos de los cuatro bloques dos equipos comparten el mismo bloque (mismas variables y mismo reto), pero cada uno trabaja un sitio o subconjunto distinto dentro del cantón Loja y al final comparan el comportamiento de sus programas sobre esos datos distintos; así el curso se vuelve un solo proyecto de investigación ambiental compartido, visto desde distintos ángulos.

Proyecto integrador – Parte 1: formulación de la investigación

Fase	Peso	Qué se evalúa
1. Formulación	3 %	Delimitación inicial del problema ambiental y del bloque de variables asignado al equipo.
2. Avance	4 %	Exploración preliminar de los datos que sustente la pregunta de investigación.
3. Producto técnico	10 %	Documento formal en LaTeX con normas APA: planteamiento del problema, pregunta de investigación, hipótesis y objetivos (general y específicos).
4. Defensa individual	8 %	Sustentación oral individual del planteamiento: problema, pregunta, hipótesis y objetivos.
Total por unidad	**25 %**	**2,50 puntos sobre 10,00 en cada unidad.**

Este producto técnico es la **Parte 1 del proyecto integrador**: la formulación completa que las unidades 2 y 3 retomarán y desarrollarán hasta el informe final.

Cronograma – Unidad 1

Semana	Fechas (mar/mié)	Contenido y proyecto integrador
1	29-sep / 30-sep	Bienvenida, hoja de ruta y formación de los 6 equipos. Entrega del dataset FIRELAB-Loja. Introducción a la programación.
2	06-oct / 07-oct	Fundamentos de algoritmos. **Fase 1 – Formulación (3 %)**: entrega 07-oct.
3	13-oct / 14-oct	Herramientas para el diseño de algoritmos. **Fase 2 – Avance (4 %)**: entrega 14-oct.
4	20-oct / 21-oct	Aplicación de algoritmos. **Fase 3 – Producto técnico (10 %)**: entrega 21-oct.
5	27-oct / 28-oct	Ingreso de datos y visualización. Síntesis. **Fase 4 – Defensa individual (8 %)** y **examen de Unidad 1**.

Pase de calificaciones de la Unidad 1: semana del 03 al 07 de noviembre de 2026.

Fechas estimadas a partir del patrón del calendario institucional 2026–2027 (mismo periodo e inicio que Matemática 2 y Estadística Descriptiva); confirmar en el SIAAF/EVA -esta asignatura no cuenta aún con reporte oficial de fechas.

Cronograma – Unidad 2

Semana	Fechas aproximadas	Contenido y proyecto integrador
1	10-nov / 11-nov	Retroalimentación del examen de Unidad 1. Condicionales.
2	17-nov / 18-nov	Funciones. **Fase 1 – Formulación (3 %)**.
3	24-nov / 25-nov	Iteraciones. **Fase 2 – Avance (4 %)**.
4	01-dic / 02-dic	Cadenas y operaciones con cadenas. **Fase 3 – Producto técnico (10 %)**.
5	08-dic / 09-dic	Listas y diccionarios. **Fase 4 – Defensa individual (8 %)**.
Receso académico estimado: 21-dic-2026 al 04-ene-2027 (sin clases).		
6	05-ene / 06-ene	Tuplas. Síntesis. **Examen de Unidad 2**.

Pase de calificaciones de la Unidad 2: semana del 11 al 15 de enero de 2027.

Cronograma – Unidad 3

Semana	Fechas aproximadas	Contenido y proyecto integrador
1	12-ene / 13-ene	Retroalimentación del examen de Unidad 2. Manejo de archivos. **Fase 1 – Formulación (3 %)**.
2	19-ene / 20-ene	Bases de datos y SQL. **Fase 2 – Avance (4 %)**.
3	26-ene / 27-ene	Transformación y agregación de dataframes. **Fase 3 – Producto técnico (10 %)**.
4	02-feb / 03-feb	Porciones indexadas de dataframes. **Fase 4 – Defensa individual (8 %)**.
5	09-feb / 10-feb	Creación y visualización de dataframes. Síntesis del curso. **Examen de Unidad 3**.

Pase de calificaciones de la Unidad 3: semana del 16 al 20 de febrero de 2027. Recuperación (quien no alcance 7/10): pase de calificaciones 08 al 12-mar-2027.

Panorama de evaluación del curso

Componente	Instrumento	Ponderación
Evidencias en aula	Micro-retos (4, 0–3) + trabajo técnico (3, 0–3) + sustentación/validación (2, 0–3).	15 % – 1,50
Lecciones	Promedio de 3 lecciones individuales (0–10).	25 % – 2,50
Proyecto integrador	Formulación 3 % + avance 4 % + producto técnico 10 % + defensa individual 8 %.	25 % – 2,50
Examen	Evaluación individual de fin de unidad.	35 % – 3,50
Total por unidad		**100 % – 10,00**

Esta estructura se repite, con los mismos pesos, en las 3 unidades (`calificaciones_programacion.xlsx`). **La calificación final es el promedio de las tres unidades.**

Nota mínima: 7/10. Celda vacía = evidencia pendiente; 0 = evidencia sin presentar. Recuperación solo si se reprueba menos del 20 % de horas y no es tercera matrícula.

Propósito de la Unidad 1

En esta unidad

Se sientan las bases del pensamiento computacional: el entorno de programación, los fundamentos de un algoritmo, las variables y expresiones, y el ingreso y despliegue de resultados, aplicados a problemas ambientales sencillos y reproducibles.

Contexto ambiental

Los ejemplos recorren relieve y accesibilidad, cobertura vegetal, clima e incendios forestales del dataset FIRELAB-Loja: los mismos contextos que un ingeniero ambiental automatiza en campo.

Resultado de aprendizaje de la unidad: conoce los elementos y estructuras básicas necesarias para el manejo de un lenguaje de programación, con capacidad de identificación y resolución de problemas sencillos, demostrando compromiso profesional.

Al finalizar la Unidad 1, usted podrá

1. Configurar y utilizar el entorno de programación en Visual Studio Code con Python.
2. Distinguir problema, algoritmo y programa, y representar un algoritmo con pseudocódigo o diagrama de flujo.
3. Declarar variables y constantes, y construir expresiones con los operadores adecuados.
4. Transformar un problema ambiental sencillo en una secuencia problema-datos-algoritmo-código-prueba-resultado.
5. Escribir programas que reciban datos con `input()` y muestren resultados legibles con `print()`.
6. Usar herramientas de inteligencia artificial de forma ética y responsable como apoyo al aprendizaje.

La IA en la programación: oportunidad y responsabilidad

Por qué importa hoy

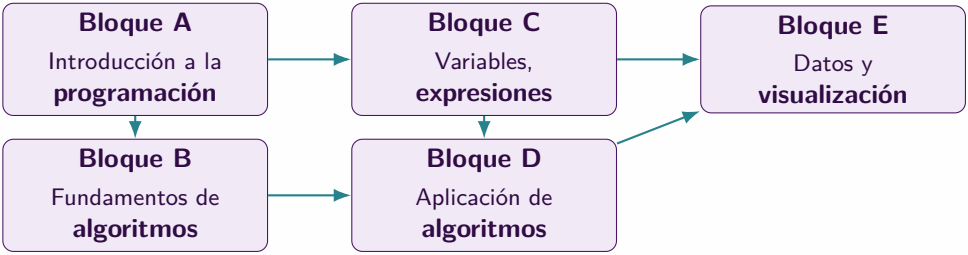
- Los asistentes de IA (Copilot, ChatGPT, Gemini, etc.) ya son parte del flujo real de trabajo de cualquier programador o ingeniero.
- Bien usados, aceleran tareas repetitivas: depurar errores, entender mensajes de error, explorar sintaxis nueva.
- No usarlos no lo hace *más riguroso*; usarlos sin entender lo que hacen sí es un riesgo real.

Su responsabilidad

- Debe poder **explicar** cada línea de código que entrega como propia, la haya escrito usted o la IA.
- Verifique siempre el resultado: la IA se equivoca con total confianza.
- Cuidado al subir datos del proyecto a herramientas de IA sin conocer su política de privacidad.

El sílabo del curso habilita el uso de IA como **apoyo**, nunca como sustituto del aprendizaje. Retomaremos esta política con ejemplos concretos de código en el Bloque A.

Mapa de contenidos



Lógica de avance: del entorno y la sintaxis (bloque A) hacia el diseño del algoritmo (B, C), su aplicación completa (D) y la comunicación de resultados (E).

Bloque A

Introducción a la programación

Entorno de trabajo, sintaxis básica y tipos de errores

Definición

Programar es escribir instrucciones, en un lenguaje que la computadora entiende, para resolver un problema. El **pensamiento computacional** es la habilidad de descomponer un problema, reconocer patrones y diseñar una solución paso a paso, antes de escribir código.

- Usaremos **Python**: un lenguaje interpretado, de sintaxis clara y muy usado en ciencia de datos ambientales.
- El entorno de trabajo del curso es **Visual Studio Code (VS Code)**.

Aprender a programar no es memorizar código: es aprender a pensar de forma estructurada y verificable.

Sintaxis básica y tipos de errores

```
# Primer programa: mensaje del proyecto FIRELAB-Loja
mensaje = "Monitoreo ambiental - canton Loja"
print(mensaje)
```

- # inicia un **comentario**: no se ejecuta, documenta el código.
- La **indentación** (espacios al inicio de línea) organiza los bloques de código en Python.

Errores de sintaxis: el código no respeta las reglas del lenguaje. **Errores de ejecución:** el programa corre pero falla a mitad de camino. **Errores lógicos:** el programa corre sin fallar, pero el resultado es incorrecto.

Uso ético de la inteligencia artificial generativa

Contexto del curso

El sílabo habilita el uso de herramientas de IA generativa como **apoyo** para revisar código, identificar errores o contrastar soluciones -nunca como sustituto de la comprensión ni de la responsabilidad del estudiante.

- Se puede usar IA para **explicar** un error o **sugerir** una mejora.
- El estudiante debe poder **explicar** cada línea que entrega como propia.
- Se valorará la integridad académica, la privacidad y el uso responsable de los datos.

Comprobación rápida: introducción a la programación

Comprobación rápida

Un programa que calcula el promedio de precipitación se ejecuta sin marcar ningún error, pero entrega un resultado que no coincide con el cálculo manual. ¿Qué tipo de error es y qué se debe revisar?

Respuesta razonada: es un **error lógico** -el programa corre, pero el procedimiento no calcula lo que se pretende-. Se debe revisar la fórmula, el orden de las operaciones y si se están incluyendo todos los datos esperados.

Síntesis del Bloque A

Programar es traducir el pensamiento computacional en instrucciones precisas; Python y VS Code son las herramientas de trabajo de este curso.

Distinguir errores de sintaxis, de ejecución y lógicos es la primera habilidad de depuración que debe dominar todo programador.

A continuación: los fundamentos de un algoritmo, antes de escribir cualquier línea de código.

Bloque B

Fundamentos de algoritmos

Problema, algoritmo y programa

Definiciones

Problema: situación que requiere solución. **Algoritmo:** secuencia finita y ordenada de pasos que resuelve el problema. **Programa:** algoritmo escrito en un lenguaje que la computadora ejecuta.

Contexto ambiental

Problema: ¿cuántos días del mes tuvieron precipitación por encima de un umbral? El algoritmo recorre cada registro diario y cuenta los que cumplen la condición.

$$\text{Entrada} \rightarrow \text{Proceso} \rightarrow \text{Salida}$$

Descomposición, abstracción y patrones

- **Descomposición:** dividir un problema grande en partes más pequeñas y manejables.
- **Abstracción:** quedarse con lo esencial del problema, ignorando detalles irrelevantes.
- **Reconocimiento de patrones:** identificar similitudes con problemas ya resueltos.

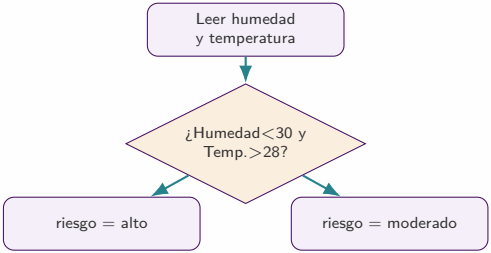
Contexto ambiental

Clasificar el riesgo de incendio de una celda se descompone en: (1) leer la humedad y la temperatura, (2) comparar contra umbrales, (3) asignar una categoría de riesgo.

Pseudocódigo y diagramas de flujo

Pseudocódigo

```
leer humedad, temperatura
si humedad < 30 y temperatura >
    28:
    riesgo = "alto"
si no:
    riesgo = "moderado"
mostrar riesgo
```



Comprobación rápida: fundamentos de algoritmos

Comprobación rápida

Un equipo debe calcular el porcentaje de celdas con cobertura de bosque nativo mayor al 50 %. ¿Cuáles serían la entrada, el proceso y la salida de ese algoritmo?

Respuesta razonada: Entrada: el listado de celdas con su porcentaje de bosque nativo. **Proceso:** contar cuántas celdas superan 50 % y dividir entre el total de celdas. **Salida:** el porcentaje resultante.

Síntesis del Bloque B

Un algoritmo es la secuencia ordenada de pasos -entrada, proceso y salida- que resuelve un problema antes de escribir cualquier código.

El pseudocódigo y los diagramas de flujo permiten diseñar y comunicar un algoritmo con claridad, independientemente del lenguaje de programación.

A continuación: las herramientas concretas -variables, expresiones y sentencias- para escribir ese algoritmo en Python.

Bloque C

Herramientas para el diseño de algoritmos

Variables, expresiones y sentencias

Variables, constantes y tipos de datos

- **Variable:** nombre que identifica un valor que puede cambiar durante la ejecución.
- **Constante:** valor que no cambia durante la ejecución del programa.
- **Identificador:** el nombre de una variable o constante (debe ser claro y descriptivo).

```
precipitacion_mm = 45.6       # float
num_estaciones = 12           # int
zona_riesgo = True            # bool
nombre_cuenca = "El Carmen"   # str
```

Python identifica el tipo automáticamente según el valor asignado: `int`, `float`, `bool` y `str` son los tipos básicos de este curso.

Operadores y expresiones

Tipo de operador	Ejemplos
Aritméticos	`+ - * / // % **` (suma, resta, producto, división, división entera, módulo, potencia)
Relacionales	`== != > < >= <=` (comparan dos valores; devuelven `True` o `False`)
Lógicos	`and or not` (combinan condiciones)

```
dias_secos = 18
dias_totales = 30
proporcion_seca = dias_secos / dias_totales  # expresion aritmetica
alerta = proporcion_seca > 0.5                # expresion relacional
```

Comprobación rápida: variables y expresiones

Comprobación rápida

¿Qué tipo de dato (`int`, `float`, `bool` o `str`) es el más adecuado para guardar el nombre de una estación meteorológica, y cuál para guardar si un día tuvo o no precipitación?

Respuesta razonada: el nombre de la estación es `str` (una cadena de texto); si hubo o no precipitación es `bool` (`True`/`False`), porque es una condición de dos valores posibles.

Síntesis del Bloque C

Las variables guardan datos con un tipo determinado; los operadores aritméticos, relacionales y lógicos construyen expresiones a partir de ellas.

Elegir bien el tipo de dato evita errores lógicos y facilita la lectura del programa.

A continuación: aplicar estas herramientas a un problema ambiental completo.

Bloque D

Aplicación de algoritmos

De un problema real a una solución computacional

La secuencia completa de resolución

Problema $\rightarrow$ Datos $\rightarrow$ Algoritmo $\rightarrow$ Código $\rightarrow$ Prueba $\rightarrow$ Resultado

- **Prueba de escritorio:** seguir el algoritmo a mano, con datos pequeños, antes de programarlo.
- **Depuración:** corregir errores identificados durante la prueba o ejecución.
- **Validación:** verificar que el resultado tiene sentido para el problema ambiental planteado.

Introducir principios de reproducibilidad: el mismo código, con los mismos datos, debe producir siempre el mismo resultado.

Ejemplo aplicado: contar eventos de incendio por celda

```
eventos = [0, 1, 0, 2, 0, 0, 1, 3, 0, 1]  # incendios registrados por celda
total_celdas_con_incendio = 0

for valor in eventos:
    if valor > 0:
        total_celdas_con_incendio = total_celdas_con_incendio + 1

print("Celdas con al menos un incendio:", total_celdas_con_incendio)
```

El algoritmo recorre cada celda (entrada), decide si tuvo incendios (proceso) y acumula el conteo (salida) -la misma lógica de "entrada-proceso-salida" del Bloque B, ya escrita en Python.

Comprobación rápida: aplicación de algoritmos

Comprobación rápida

Antes de ejecutar el programa del ejemplo anterior, ¿qué resultado debería arrojar una prueba de escritorio con la lista `[0,0,2,0,1]`?

Respuesta razonada: 2 celdas con al menos un incendio (las que valen 2 y 1); los tres ceros no cumplen la condición `valor > 0`. Si el programa no arroja 2, hay un error que depurar.

Síntesis del Bloque D

Todo problema se resuelve siguiendo la secuencia problema-datos-algoritmo-código-prueba-resultado.

La prueba de escritorio y la depuración son parte del proceso, no un paso opcional al final.

A continuación: cómo ingresar datos y comunicar resultados de forma clara y accesible.

Bloque E

Ingreso de datos y visualización

`input()`, `print()` y comunicación de resultados

Entrada y salida de información

```
nombre_zona = input("Nombre de la zona de monitoreo: ")
temperatura = float(input("Temperatura registrada (C): "))

print(f"Zona: {nombre_zona} - Temperatura: {temperatura:.1f} C")
```

- `input()` siempre entrega texto (`str`); si se necesita un número, debe **convertirse** con `float()` o `int()`.
- Las **f-strings** (`f"..."`) insertan variables y dan formato dentro de un mensaje.

Validar el dato ingresado (por ejemplo, que la temperatura sea un número) evita errores de ejecución más adelante en el programa.

Comunicación clara y accesible de resultados

- Los mensajes de salida deben ser **legibles**: unidades, decimales razonables y contexto.
- El formato `f"valor:.1f"` controla cuántos decimales se muestran.
- Presentar resultados de forma clara favorece la **accesibilidad** de la información para cualquier lector.

Contexto ambiental

Un reporte que dice "23.456789 C" es menos útil que uno que dice "23.5 °C" -el segundo comunica con la precisión que el problema realmente necesita.

Comprobación rápida: datos y visualización

Comprobación rápida

Un programa recibe con `input()` el número de días con precipitación, pero al sumarlo con otro número marca un error. ¿Cuál es la causa más probable?

Respuesta razonada: `input()` devuelve el dato como **texto (`str`)**, no como número; falta convertirlo con `int()` (o `float()`) antes de sumarlo.

Síntesis del Bloque E

`input()` y `print()` son la interfaz básica entre el programa y quien lo usa; conversión de tipos y f-strings hacen esa interfaz confiable y clara.

Comunicar un resultado con el formato adecuado es tan importante como calcularlo correctamente.

A continuación: cierre de la Unidad 1 y enlace con condicionales e iteraciones en la Unidad 2.

Cierre

Síntesis integral de la Unidad 1

Del entorno de trabajo a un primer programa completo

Lo que hemos construido

1. **Bloque A:** el entorno de trabajo (VS Code, Python) y la sintaxis básica del lenguaje.
2. **Bloque B:** problema, algoritmo y programa; pseudocódigo y diagramas de flujo.
3. **Bloque C:** variables, tipos de datos y expresiones con operadores.
4. **Bloque D:** la secuencia completa problema-datos-algoritmo-código-prueba-resultado.
5. **Bloque E:** entrada con `input()` y salida clara con `print()` y f-strings.

Ejercicio de cierre: FIRELAB-Loja

Un equipo necesita un programa que pida al usuario la cantidad de hectáreas quemadas en un evento y muestre si supera un umbral de alerta de 50 hectáreas.

Comprobación rápida

¿Qué bloques de esta unidad debe combinar el equipo para escribir ese programa? Use lo aprendido en los bloques C, D y E.

Respuesta razonada: Bloque E para leer el dato con `input()` y convertirlo a número; **Bloque C** para guardar el umbral en una variable y construir la expresión relacional de comparación; **Bloque D** para seguir la secuencia completa -incluida una prueba de escritorio- antes de confiar en el resultado.

Cierre de la Unidad 1

Programar es traducir un problema ambiental en una secuencia precisa de pasos, y esos pasos en código que se puede probar, depurar y confiar.

Las variables, expresiones y la entrada/salida de esta unidad son la base de las estructuras de control que veremos en la Unidad 2.

Siguiente paso: condicionales e iteraciones en la Unidad 2